

ARK-499 → ARK-503 — Enterprise Adapter & Operational Readiness Series

Frozen Preregistration v1.0

Program: ExecutionProof™ · Verification Before Execution™ · Proof Before Power™

Author / Sole Inventor: Derek Adam Hone — Remnant Fieldworks Inc.

Date frozen: 2026-07-20

Series ID: ARK-499-503

Standing corpus at time of writing: 72 preregistered experiments · 232 case records · 229 PASS · 2 preserved FAIL · 1 preserved GATE-STOP · 9 public repositories.

0. Purpose and honest scope

The prior series (ARK-493–498) established, within self-contained synthetic testbeds, that the ExecutionProof gate makes correct decisions, enforces (not merely logs) them, binds approvals to exact actions, re-resolves authority at execution time, blocks self-approval, fails closed under dependency loss, and produces independently verifiable ProofRecords.

This series tests a different and previously-unanswered question: **can the SAME unmodified gate control real, externally-maintained enterprise interfaces using authentic protocols, credential models, state sources, and side-effect systems — not mocks?**

The gate is frozen. ARK-499–503 reuse the byte-identical `gate/` and `guards/` modules from the ARK-493–498 testbed (SHA-256 of each file is recorded in the manifest). No decision logic is added or altered for this series. What changes is only the **enforcement adapter**: on ALLOW it now performs a real side effect against a real external system; on DENY/HOLD it performs zero side effects. State is then **independently inspected in the real external system**, not asserted by the testbed.

0.1 Honest feasibility declaration (locked before execution)

This VM is ephemeral (it halts on idle) and has no running container daemon. Therefore this preregistration commits, in advance, to the following honesty boundaries — these are not post-hoc excuses:

- **ARK-499, ARK-500, ARK-501 are fully executable here** using real native components (a real PostgreSQL 17 cluster; a real git repository + a real local CI runner acting on real build artifacts with real SHA-256 digests; a real standards-based RS256 JWT/JWKS identity issuer served over HTTP). These produce measured, hash-chained, dual-guard-verified ProofRecords.
- **ARK-502 (endurance) is PARTIAL by design on this host.** A true ≥ 14 -day endurance result is **impossible on an ephemeral VM** and is NOT claimed. This series builds and runs the endurance **harness** plus a bounded **smoke run**. The smoke run validates that the harness exercises the required stressors and that the ProofRecord chain stays continuous across restarts. The defensible ≥ 14 -day run must be executed on a persistent machine; until then no endurance claim is made.
- **ARK-503 (independent enterprise review) CANNOT be self-certified.** By definition it requires a

human reviewer who did not build the experiment. This series produces the **reviewer package** (installable adapter bundle, tasks, and a scored rubric). The experiment is marked **NOT-EXECUTED — AWAITING INDEPENDENT REVIEWER** and contributes zero PASS to the corpus until a human completes it.

No result in this series asserts production certification. Bounded claims only.

1. Reused frozen components (the gate under test)

The following files are copied byte-for-byte from `ark-493-498-testbed` and their SHA-256 is locked in `PREREGISTRATION-MANIFEST.txt` :

- `gate/core.py` , `gate/gate.py` , `gate/policy.py` , `gate/actor_registry.py`
- `guards/guard_a.py` , `guards/guard_b_verifier.py`

Canonical serialization, ed25519 signing (testbed key seed `b"\x00"*32` — testbed-only, NOT a production secret), SHA-256 hashing, the six gate dimensions, and the dual-guard verification are all unchanged. The ProofRecord schema is `ark-enterprise-proofrecord-v1.0` ; the policy version is `ark-enterprise-v1.0` .

2. Common method for all executable experiments

1. Build the shared environment: the frozen registry, policy store, gate, dual-guard proof store, and the experiment-specific **real adapter**.
2. For each preregistered case, the actor agent constructs a canonical action request (exact-action hash, evidence, idempotency key) exactly as in ARK-493-498.
3. The **real enforcement point** calls the frozen gate synchronously, fails closed on any error, and:
 - **ALLOW** → performs the real side effect against the external system **exactly once**, records the real system's own identifiers (row id, artifact digest, resource state), sets `tool_called=true` .
 - **DENY / HOLD** → performs **zero** side effects, sets `tool_called=false` .
4. Every action produces a signed, hash-chained ProofRecord verified by Guard-A (in-process) and Guard-B (isolated subprocess). Dual-guard agreement is required.
5. After each experiment, an **independent inspector** queries the real external system directly (a fresh DB connection / a fresh read of the deploy target / an independent token validation) and the measured external state is compared to the ProofRecord ledger. Any divergence is a leak.

Enforcement-leak definition (series-wide): any real external state change that occurred without a corresponding ALLOW ProofRecord, or any ALLOW that produced either zero or more than one real state change. Leak count MUST be zero.

3. ARK-499 — Real PostgreSQL Transaction Boundary

New question: Can ExecutionProof prevent an actual unauthorized database state change through a real database protocol (not classification of a mock destructive action)?

Real component: a freshly-initialized native PostgreSQL 17 cluster on a private port, with a real application schema, two real database roles (a privileged writer the adapter uses, and a read-only `ark_auditor` used only for independent inspection), real transactions, real `COMMIT / ROLLBACK`, and real connection loss.

Arms (preregistered):

- **A1 Authorized write (ALLOW):** authorized actor + fresh evidence + exact-hash match → the adapter opens a real transaction, performs the real `INSERT/UPDATE`, commits exactly once.
- **A2 Unauthorized write (DENY):** actor lacking DB authority → zero rows changed.
- **A3 Mutated action (DENY):** approved hash pinned to a different row/value than presented → zero rows changed.
- **A4 Stale-authority write (DENY):** authority revoked after approval, before execution → zero rows.
- **A5 Missing-evidence write (HOLD):** required evidence absent → zero rows, non-executable.
- **A6 Mid-transaction failure (fail-closed):** dependency (DB connection) dropped during an otherwise ALLOW action → transaction rolls back, zero rows committed, ProofRecord shows fail-closed DENY.
- **A7 Concurrent idempotency:** the same authorized write submitted N times concurrently under one idempotency key → exactly one row committed.

Metrics: rows_committed per case (measured by the independent `ark_auditor` role); leak count; dual-guard agreement; exactly-once count for A7.

Thresholds (hard): P-499-1 zero unauthorized/denied/held writes reach the DB (leak=0); P-499-2 every ALLOW commits exactly one change; P-499-3 A6 rolls back with zero committed rows; P-499-4 A7 commits exactly once under concurrency; P-499-5 dual-guard agreement on 100% of records.

Kill condition: if any DENY/HOLD case produces a committed row, ARK-499 is EXPERIMENT-FAIL and the FAIL is preserved (not rewritten).

4. ARK-500 — Real CI/CD Release Boundary

New evidence sought: only the exact approved artifact reaches the exact approved environment, enforced through a real release runner acting on real files with real digests.

Real component: a real local git repository, a real build step producing a real artifact tarball with a real SHA-256 digest, and a real “runner” that promotes an artifact into a real `environments/` deploy target directory on disk (staging, production). Independent inspection reads the actual deployed files and recomputes their digests. (Explicitly NOT claimed: Docker/Kubernetes/cloud CD — a local runner and on-disk deploy target only.)

Arms (preregistered):

- **B1 Approved release (ALLOW):** approved artifact digest + approved environment → the runner deploys exactly that artifact to exactly that environment; deployed digest matches.
- **B2 Artifact-digest substitution (DENY):** a different artifact than the approved digest → zero deployment.

- **B3 Environment substitution (DENY):** approved for staging, attempted against production → zero deployment to production.
- **B4 Stale scan evidence (HOLD):** required scan evidence stale → non-executable, zero deployment.
- **B5 Revoked reviewer authority (DENY):** reviewer authority revoked before execution → zero deploy.
- **B6 Retry (idempotency):** approved release retried under one key → deployed exactly once.
- **B7 Concurrent deploy triggers:** same approved release fired concurrently → deployed exactly once.

Metrics: deployed artifact digest per environment (measured from disk); count of deployments; leak count; dual-guard agreement.

Thresholds (hard): P-500-1 only the approved digest reaches the approved environment; P-500-2 zero deployments for all DENY/HOLD arms; P-500-3 exactly-once under retry/concurrency; P-500-4 dual-guard agreement 100%.

Kill condition: any wrong-artifact or wrong-environment deployment → EXPERIMENT-FAIL, preserved.

5. ARK-501 — Enterprise Identity Adapter

New question: Can the boundary resolve current authority from an external identity source (standards-based signed tokens) rather than internally-generated fixtures?

Real component: a real RS256 JWT issuer with a real JWKS endpoint served over HTTP (loopback), issuing real signed access tokens carrying `sub`, `roles / groups`, `exp`, and a `jti`; a real token revocation list; real key material. An identity-adapter shim resolves authority for the gate by **validating the real token signature against the fetched JWKS, checking real `exp` expiry, checking**

the revocation list, and mapping token roles to tool authority — i.e. authority comes from the external issuer, not the in-memory registry.

Arms (preregistered):

- **C1 Valid token, sufficient role (ALLOW):** real signed unexpired token with the required role.
- **C2 Expired token (DENY):** real token past `exp` → denied.
- **C3 Bad signature / wrong key (DENY):** token signed by a non-issuer key → denied.
- **C4 Insufficient role (DENY):** valid token lacking the required role/group → denied.
- **C5 Revoked token (DENY):** valid unexpired token whose `jti` is on the revocation list → denied.
- **C6 Revocation after issue, before execution (DENY):** token valid at issue, revoked before the execution attempt → authority re-resolved at execution time → denied.
- **C7 Group-membership escalation (DENY):** token requests a tool above its group's mapped authority → denied.

Metrics: decision per arm; whether authority was resolved from the external issuer (evidenced by a recorded JWKS key id + `jti`); leak count; dual-guard agreement.

Thresholds (hard): P-501-1 only C1 authorizes; P-501-2 all of C2-C7 denied with zero side effects; P-501-3 authority provably resolved from the external issuer (kid/jti recorded); P-501-4 dual-guard agreement 100%.

Kill condition: any expired/revoked/insufficient/forged token authorizing a side effect → EXPERIMENT-FAIL, preserved.

6. ARK-502 — Long-Running Reliability & Restart Safety (PARTIAL on this host)

Honest status: a defensible endurance claim requires **≥14 days** including planned/unplanned restarts, policy-version changes, key rotation, database migration, queue recovery, dependency outages, clock drift, malformed requests, operator configuration mistakes, rolling deployment, and ProofRecord-chain continuity. **This cannot be run on an ephemeral VM and is NOT claimed here.**

What this series delivers for ARK-502:

- A real endurance **harness** that drives continuous authorized/unauthorized traffic through the gate and real adapter, injecting the stressor set above on a schedule.
- A bounded **smoke run** (minutes, not days) that exercises at least: a process restart with chain resume, a policy-version change, a key-rotation event, a dependency outage + recovery, and malformed requests — proving the harness works and the ProofRecord chain remains continuous and dual-guard clean across a restart.

Metrics (smoke): chain continuity across restart (prior-hash linkage unbroken); leaks during outage windows (must be 0); dual-guard agreement; count of stressor events exercised.

Thresholds: P-502-SMOKE-1 chain continuity preserved across ≥ 1 restart; P-502-SMOKE-2 zero leaks during injected outages; P-502-SMOKE-3 dual-guard agreement 100%.

Reported claim ceiling: “survived a bounded production-like smoke run with chain continuity across restart” — explicitly NOT “weeks” and NOT “production.” The ≥ 14 -day run is deferred to a persistent machine and recorded as OPEN.

7. ARK-503 — Independent Enterprise Review Challenge (NOT self-executable)

Status: NOT-EXECUTED — AWAITING INDEPENDENT REVIEWER. Contributes zero PASS until a human who did not build the experiment completes it.

Deliverable of this series: a reviewer package containing the installable adapter bundle, documentation, and a scored task list: install from documentation, configure a policy, rotate keys, add a new action type, diagnose a HOLD, verify ProofRecords with the isolated verifier, recover from a simulated outage, reproduce outcomes, inspect the real downstream systems, and attempt bypasses. The rubric records each task PASS/FAIL and reviewer independence (Adith = internal independence; an outside reviewer is stronger; an unassisted customer would be the real answer).

8. Publication rule

Publish every outcome — PASS, FAIL, HOLD, GATE-STOP, PARTIAL, and NOT-EXECUTED — without re-writing.

Report ARK-499/500/501 as bounded integration results in the tested environments only. Report ARK-502

as a harness + smoke result, never as an endurance claim. Report ARK-503 as pending independent human

review. Keep every claim narrower than the evidence. Make no “world first” without a formal prior-art search. Maintain the standing disclosures (device-dependence for quantum work is N/A here;

tamper-evident $\neq$ unforgeable; engineering tests $\neq$ production certification; Zenodo archival $\neq$ peer review; RF-100 not NIST-endorsed or legally binding).

When the narrative and the machine record disagree, the machine record governs — even against the founder.

9. Corpus counting rule (locked)

ARK-499/500/501 scored cases are counted as **individual case records** (consistent with the ARK-493-498

individual-cell counting method). ARK-502 smoke and ARK-503 package contribute **zero** scored PASS records to the public corpus headline until the ≥ 14 -day run and the human review, respectively, are completed on appropriate infrastructure. The corpus headline must disclose the mixed counting method.

Built in faith. Tested in public. Claims kept narrower than the evidence.

Proof Before Power™ · Verification Before Execution™